Methodology Report

1. Preprocessing

The audio files were first loaded at a fixed sampling rate of 16 kHz and converted to mono format. Since the model requires inputs of the same length, each audio sample was converted into a fixed duration of 2 seconds, corresponding to 32000 audio samples. Audio files longer than this length were trimmed, while shorter files were padded with zeros.

This preprocessing step ensured that all input audio samples had a consistent shape before being passed to the feature extraction stage.

2. Feature Extraction

Raw audio is a one-dimensional waveform, which represents amplitude changes over time. To make it suitable for CNN-based classification, each waveform was converted into a two-dimensional spectrogram using Short-Time Fourier Transform (STFT).

The spectrogram represents how frequency components change over time. Log scaling was applied to the spectrogram values to reduce the effect of very large magnitude differences. The spectrogram was then resized to 128 × 128 and standardized using per-image standardization. This converted each audio sample into an image-like input suitable for a CNN model.

The final input shape used by the model was:

128 × 128 × 1

where 1 represents the single-channel spectrogram.

3. Model Architecture

A Convolutional Neural Network (CNN**)** was used for binary classification of audio samples into Genuine (Human) and Deepfake (AI-Generated) classes. The CNN was chosen because spectrograms behave like image representations of audio signals, allowing convolutional layers to learn useful time-frequency patterns.

The model architecture consisted of multiple convolutional blocks. Each block included convolutional layers for feature extraction, batch normalization for stable training, ReLU activation for non-linearity**, max pooling** for dimensionality reduction, and spatial dropout**-**based regularization to reduce overfitting.

After feature extraction, global max pooling was used to convert the learned feature maps into a compact representation. Dense layers were then used for final classification, and the output layer used a softmax activation with two neurons corresponding to the two classes: fake and real.

4. Final Prediction Strategy

The model outputs probability scores for both classes: Deepfake (AI-Generated) and Genuine (Human). Instead of using the default threshold of 0.5, the final prediction is made using the selected fake-class decision threshold(.

If the fake-class probability is greater than or equal to the selected threshold, the audio is classified as Deepfake (AI-Generated). Otherwise, it is classified as Genuine (Human).

A threshold-aware confidence score is also calculated to show how far the prediction is from the decision boundary. For deepfake predictions, confidence is calculated as:

confidence = (fake_probability - threshold) / (1 - threshold)

For genuine predictions, confidence is calculated as:

confidence = (threshold - fake_probability) / threshold

The confidence score is clipped between 0 and 1 and displayed as a percentage in the Python script and Streamlit application.